

Error Handling Documentation – Nairobi Nexus Newsletter System

1. Error Detection Techniques

Technique	Where Implemented	How It Detects (from JSON)
HTTP <code>continueErrorOutput</code>	All 5 HTTP Request nodes	Each HTTP node has <code>"onError":</code> <code>"continueErrorOutput"</code> (e.g., <code>HTTP _ TechCabal</code> , <code>HTTP_Business Daily Reporter</code> , etc.). If a source fails, the node outputs an error item to its orange output, but the workflow continues.
Truncation Code node	After each HTTP node	Code nodes (e.g., <code>Truncate TechCabal</code>) reduce HTML to ≤ 5000 characters, preventing out-of-memory errors before they happen.

Defensive validation	After each Field Reporter	Code nodes (e.g., <code>Validate</code>
Code node	AI	<code>TechCabal</code>) check <code>item.json.output</code> for <code>source</code> and <code>headline</code> . If missing, they set <code>valid: false</code> and return a safe object.

Output parser validation	All AI agents	Each Field Reporter, Chief Editor, Backup, QA, and Audit AI has a <code>Structured Output Parser</code> that rejects malformed JSON. If the parser fails, the AI node can trigger its error output (if configured).
--------------------------	---------------	---

Primary Chief Editor

`onError`

`AI Agent_Chief Editor`

Has `"onError" :`

`"continueErrorOutput"` (in

JSON: `"onError" :`

`"continueRegularOutput"`

? Actually check: in your main workflow JSON, the primary Chief Editor does not have an explicit `onError` field – but in the backup test workflow you set `maxTokens:0` to force failure. For production, you have `retryOnFail: true` but no `onError`. However, the orange output connection exists because you manually connected it. The orange output is activated only when the node fails (e.g., due to API error). You rely on the fallback path.

2. Error Recovery Mechanisms

Error scenario	Recovery action (as per your JSON connections)
Primary Chief Editor fails	The orange error output of <code>AI Agent_Chief Editor</code> is connected to <code>Backup Chief Editor</code> . The backup uses <code>magistral-small-latest</code> model with a simpler prompt (under 80 words, basic slang, apology). After the backup runs, <code>Flag Fallback</code> Code node adds <code>_fallback = true</code> .
Field Reporter fails	The validation Code node marks the item as <code>valid: false</code> . The <code>Count & Governance</code> node counts only <code>valid: true</code> items, so the failed source is excluded from the <code>sourcesSucceeded</code> count but listed in <code>sourcesFailedList</code> . The Merge and Aggregate nodes still receive an item (with <code>valid: false</code>), so the workflow does not break.
Chief Editor output empty	<code>Validate Chief Output</code> Code node checks if <code>output.intro</code> exists; if not, it inserts a placeholder: " <code>⚠ Newsletter generation encountered a technical issue...</code> ".

QA Agent fails (e.g., malformed JSON)	The QA Agent's output parser may fail, but in your workflow the QA Agent is not connected to an error output. If it fails, the workflow stops. To mitigate, you have set the QA Agent's model to <code>gpt-4o-mini</code> with <code>json_object</code> response format (in your main workflow JSON, QA Agent uses OpenAI Chat Model with <code>responseFormat: "json_object"</code>). This reduces the chance of malformed output.
HTTP source fails	The branch continues with no data; the validation Code node for that branch receives an error item and produces <code>valid: false</code> . The <code>Count & Governance</code> node logs the source as failed.

3. Fallback Processes

- Primary → Backup Chief Editor
 - The primary `AI Agent_Chief Editor` does not have an explicit `onError` in the JSON, but you have manually connected its orange output to `Backup Chief Editor` (see connections in the JSON: `"AI Agent_Chief Editor": { "main": [...], "error": [[{ "node": "Backup Chief Editor", ... }]] }`).
 - The backup is triggered only when the primary fails. After the backup, a `Flag Fallback Code` node adds `_fallback = true`.
 - The `Extract Governance & Intro Code` node checks `item._fallback` and sets `governance.fallbackTriggered = true`.
- QA Agent correction
 - The QA Agent's system message instructs it to correct the intro (truncate length, add slang, fix Markdown). It outputs `corrected_intro`. The `Extract Governance & Intro` node uses `corrected_intro` as the final intro.

4. Continue-on-Error Implementations

Node type	Setting (from JSON)
HTTP Request	<code>"onError": "continueErrorOutput"</code> (all 5 HTTP nodes)
Field Reporter AI	<code>"onError": "continueErrorOutput"</code> (each has <code>"onError": "continueErrorOutput"</code>)
Primary Chief Editor	No explicit <code>onError</code> ; but the orange output is connected, so any failure will trigger fallback (assuming the node fails, e.g., API error). For safety, you could add <code>"onError": "continueErrorOutput"</code> but it's not in the JSON.
Backup Chief Editor	No <code>onError</code> (default stop).
QA Agent	No <code>onError</code> (default stop).
Audit AI Agent	No <code>onError</code> (default stop).

5. Notification and Alerting Strategies

Notification	Trigger	Method (from JSON)	Recipient
Newsletter draft	Successful completion of main workflow	Gmail node <code>Send a message</code> in main workflow. Subject: <code>Nairobi Nexus Draft - {{ \$now.toFormat('yyy-MM-dd') }}</code> . Body: HTML with intro and parting word.	<code>amosmokua3@gmail.com</code> (test; can be changed)
Error workflow alert	Any fatal error in main workflow (if error workflow is linked)	Error workflow (<code>Newsletter Error Handler</code>) – Gmail node with subject <code>⚠ Nairobi Nexus Newsletter – Workflow Failed at {{ \$json.execution.lastNodeExecuted }}</code> , body with AI-generated hypothesis and action.	<code>amosmokua3@gmail.com/mercykihugu484@gmail.com</code>

Audit answer	Manager calls audit	Audit workflow	amosmokua3@gmail.
	webhook	Gmail node with	com/mercykihugu48
		subject Audit	4@gmail.com
		Answer: What	
		happened last	
		Friday?, body as	
		AI-generated	
		sentence.	

6. Retry and Escalation Logic

- No explicit retry loops in your JSON. The workflow is stateless.
- Fallback acts as a “silent retry” – the backup Chief Editor runs immediately after primary failure.
- Escalation – the global error workflow sends an email to the ops team for any fatal error.

7. Logging for Debugging

All errors and fallback events are logged in the Governance sheet via the Log Governance node. Columns include:

- Fallback Triggered – set by Extract Governance & Intro based on _fallback flag.
- QA Issues – array of issues from QA Agent.
- Sources Failed – list from Count & Governance.
- Model Used – dynamic: magistral-small-latest if fallback triggered, else llama-3.3-70b-versatile.